

ASISTENCIA IA

Manual técnico, funcional y de implementación

Monitor Inteligente de Asistencia con IA

Arquitectura: Frontend + FastAPI + MySQL

Roles: Estudiante · Profesor · Administrador

IA: Chatbot contextual · Reportes · Voz · Reconocimiento facial

Versión: MySQL 1.0 · Octubre 2026

PROPÓSITO Guía maestra para construir el sistema desde cero utilizando MySQL como base de datos principal. Incluye instalación, creación del esquema, conexión con FastAPI, autenticación JWT, roles, chatbot con restricciones, reportes, voz, reconocimiento facial y pruebas.

Índice general

- 1. Visión del sistema y alcance
- 2. Arquitectura general con MySQL
- 3. Roles, permisos y reglas de negocio
- 4. Estructura completa de carpetas
- 5. Preparación del entorno desde cero
- 6. Instalación y configuración de MySQL
- 7. Diseño completo de la base de datos MySQL
- 8. Conexión FastAPI ↔ MySQL
- 9. Autenticación con JWT y contraseñas seguras
- 10. API y backend por módulos
- 11. Frontend por rol
- 12. Módulo de asistencia
- 13. Reconocimiento facial y liveness
- 14. Chatbot IA tipo Gemini y restricciones
- 15. Reportes Excel/PDF desde el chatbot
- 16. Asistente de voz
- 17. Estadística, analítica y ML
- 18. Adaptación a empresas u otros rubros
- 19. Puesta en marcha paso a paso
- 20. Pruebas funcionales y de seguridad
- 21. Copias de seguridad y mantenimiento MySQL
- 22. Errores frecuentes y soluciones
- 23. Hoja de ruta
- 24. Anexos: requirements, .env, SQL y endpoints

1. Visión del sistema y alcance

ASISTENCIA IA es una plataforma web para controlar, monitorear y analizar asistencia. El escenario inicial es educativo: estudiantes, profesores, cursos, horarios, sesiones y registros de asistencia. La arquitectura se diseña para que en una fase posterior pueda adaptarse a empresas, centros de capacitación u otras organizaciones sin reescribir el núcleo del sistema.

PRINCIPIO CENTRAL Cada usuario consulta únicamente la información permitida por su rol y por su relación con los datos. Las restricciones deben validarse en FastAPI y en las consultas SQL; no basta con ocultar botones en el frontend.

1.1 Objetivos funcionales

- Autenticar usuarios con correo y contraseña usando hashes seguros y tokens JWT.
- Gestionar los roles ESTUDIANTE, PROFESOR y ADMINISTRADOR.
- No permitir registro público: el profesor crea estudiantes; el administrador crea estudiantes y profesores.
- Registrar asistencia manual o mediante reconocimiento facial y verificación de presencia real (liveness).
- Mostrar dashboards distintos según rol.
- Calcular porcentajes, tardanzas, ausencias, promedios, medianas, tendencias y alertas.
- Proporcionar un chatbot contextual tipo Gemini, con panel lateral y modo pantalla completa.
- Permitir que el chatbot genere reportes Excel/PDF dentro del alcance del usuario.
- Incorporar entrada por voz y lectura de respuestas.
- Permitir al administrador configurar la plataforma para institución educativa, empresa o modo personalizado.

2. Arquitectura general con MySQL

La solución se divide en capas. El navegador muestra la interfaz, FastAPI concentra autenticación, reglas de negocio y autorización, y MySQL almacena los datos. Los servicios de IA se conectan al backend, no directamente al navegador.

```
NAVEGADOR (HTML/CSS/JS)
  |
  | HTTPS / JSON / JWT
  v
FASTAPI
|-- Auth + roles
|-- Cursos / usuarios
|-- Asistencia
|-- Chatbot / reportes / voz
|-- Reconocimiento facial
  |
  | SQLAlchemy + PyMySQL
  v
MYSQL
usuarios, estudiantes, profesores, cursos,
```

matriculas, horarios, sesiones_asistencia,
asistencias, datos_faciales, reportes, auditoria

Capa	Responsabilidad	Tecnologías
Frontend	Pantallas, cámara, gráficos, chatbot, formularios y voz.	HTML, CSS, JavaScript, Font Awesome, Web APIs
API	Autenticación, autorización, reglas de negocio, validación y endpoints.	FastAPI, Pydantic, Uvicorn
Datos	Persistencia transaccional.	MySQL 8.x, SQLAlchemy, PyMySQL
Seguridad	JWT, hash de contraseñas, control por rol y filtros SQL.	python-jose / PyJWT, bcrypt/passlib
IA	Reconocimiento facial, liveness, ML y chatbot.	OpenCV, TensorFlow/ONNX, scikit-learn, proveedor LLM opcional
Reportes	Excel/PDF según permisos.	openpyxl, pandas, ReportLab
Voz	Dictado y lectura de respuestas.	Web Speech API o STT/TTS externo

2.1 Flujo autenticado

```
Login -> FastAPI -> verifica password_hash -> genera JWT
-> Frontend guarda token
-> Authorization: Bearer <token>
-> FastAPI identifica usuario + rol
-> servicio aplica scope
-> consulta MySQL parametrizada
-> respuesta JSON
```

3. Roles, permisos y reglas de negocio

Función	Estudiante	Profesor	Administrador
Ver perfil propio	Sí	Sí	Sí
Ver otro estudiante	No	Solo si pertenece a sus cursos	Sí, para gestión autorizada
Ver profesor	Solo datos públicos de su curso	Solo datos necesarios	Sí
Ver datos administrativos	No	No	Sí
Crear estudiante	No	Sí	Sí
Crear profesor	No	No	Sí

Función	Estudiante	Profesor	Administrador
Crear administrador	No	No	Solo procedimiento controlado
Consultar asistencia	Solo propia	Solo alumnos de sus cursos	Global
Generar reportes	Personal	De sus cursos	Global/autorizado
Configurar sistema	No	No	Sí
Chatbot	Solo sus datos	Sus cursos y alumnos	Visión global autorizada

3.1 Creación de cuentas

- No existe botón público “Registrarse”.
- El profesor puede crear únicamente usuarios con rol ESTUDIANTE.
- El administrador puede crear ESTUDIANTE y PROFESOR.
- La creación de ADMINISTRADOR se realiza por un procedimiento administrativo separado.
- El backend vuelve a validar la regla aunque alguien manipule el HTML o envíe peticiones manuales.

```
if current_user.rol == "PROFESOR" and payload.rol != "ESTUDIANTE":
    raise HTTPException(status_code=403,
                        detail="El profesor solo puede crear estudiantes")
```

4. Estructura completa de carpetas

```
monitor_asistencia_ia/
├── .env/
├── backend/
│   ├── .env
│   ├── .env.example
│   ├── requirements.txt
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   │   ├── auth.py
│   │   │   ├── users.py
│   │   │   ├── courses.py
│   │   │   ├── attendance.py
│   │   │   ├── chatbot.py
│   │   │   └── reports.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── database.py
│   │   │   ├── security.py
│   │   │   └── auth_dependencies.py
│   │   └── models/
│   │       ├── user.py
│   │       └── student.py
```

```

├── professor.py
├── course.py
├── attendance.py
├── schemas/
├── services/
│   ├── attendance_service.py
│   ├── chatbot_policy_service.py
│   ├── chatbot_data_service.py
│   ├── chatbot_llm_service.py
│   ├── report_service.py
│   └── face_service.py
├── tests/
├── database/
│   ├── 01_create_database.sql
│   ├── 02_schema.sql
│   ├── 03_indexes.sql
│   ├── 04_seed_demo.sql
│   └── backup/
├── frontend/
│   ├── login/
│   ├── estudiante/
│   ├── profesor/
│   ├── administrador/
│   ├── components/chatbot/
│   └── assets/
│       ├── css/
│       ├── js/
│       └── img/
├── ai/
│   ├── face/
│   ├── liveness/
│   ├── attendance_model/
│   └── notebooks/

```

4.1 Archivos clave

Archivo	Función
backend/app/main.py	Crea FastAPI, configura CORS y registra routers.
core/database.py	Crea engine SQLAlchemy, SessionLocal y get_db().
core/security.py	Hash, verificación de contraseña y JWT.
core/auth_dependencies.py	Obtiene usuario actual y valida roles.
api/auth.py	Login, refresh opcional y /me.
api/users.py	Creación protegida de estudiantes y profesores.
api/chatbot.py	Endpoint del asistente.
services/chatbot_policy_service.py	Aplica restricciones por rol y scope.
database/02_schema.sql	Crea las tablas y relaciones.

Archivo	Función
frontend/assets/js/api.js	Cliente fetch común que adjunta JWT.
frontend/assets/js/chatbot.js	Interfaz, mensajes y comunicación con backend.

5. Preparación del entorno desde cero

5.1 Software necesario

- Python 3.11 o 3.12 recomendado para mayor compatibilidad con librerías de IA.
- Visual Studio Code.
- MySQL Server 8.x.
- MySQL Workbench.
- Git (opcional, recomendado).
- Google Chrome o Microsoft Edge para pruebas de cámara y voz.

5.2 Crear el entorno virtual

```
cd C:\ruta\del\proyecto
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

VERIFICACIÓN La terminal debe comenzar con (.venv). Si no aparece, las dependencias pueden instalarse en otro Python y provocar errores como ModuleNotFoundError.

5.3 Instalar dependencias base

```
cd backend
python -m pip install --upgrade pip
python -m pip install fastapi "uvicorn[standard]" sqlalchemy pymysql python-dotenv pydantic
pydantic-settings python-jose[cryptography] passlib[bcrypt] python-multipart
```

6. Instalación y configuración de MySQL

6.1 Instalar MySQL Server y Workbench

1. Descargar MySQL Installer Community para Windows desde el sitio oficial de MySQL.
2. Ejecutar el instalador y elegir Developer Default. Si esa opción no aparece, instalar al menos MySQL Server y MySQL Workbench.
3. Configurar el servidor como Development Computer.

4. Mantener TCP/IP habilitado y puerto 3306, salvo que exista un conflicto local.
5. Crear la contraseña del usuario root y guardarla de forma segura.
6. Mantener el servicio de Windows activo para que MySQL pueda iniciarse automáticamente.
7. Abrir MySQL Workbench y conectarse a la instancia local.

6.2 Prueba inicial

```
SELECT VERSION();
```

Si Workbench devuelve una versión, el servidor está funcionando.

6.3 Crear la base

```
CREATE DATABASE IF NOT EXISTS monitor_asistencia_ia
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE monitor_asistencia_ia;
SELECT DATABASE();
```

7. Diseño completo de la base de datos MySQL

A diferencia de Supabase, MySQL no administra automáticamente usuarios ni JWT. Por eso la tabla usuarios almacena el hash de la contraseña, nunca la contraseña en texto plano. La autorización se implementa en FastAPI.

7.1 Modelo lógico

```
usuarios
├── estudiantes
│   ├── matriculas
│   ├── asistencias
│   └── datos_faciales
├── profesores
│   └── cursos <──┐
│                 ├── horarios
│                 ├── sesiones_asistencia
│                 └── asistencias
```

usuarios → chatbot_historial / reportes_generados / auditoria

7.2 Tablas, claves y propósito

Tabla	PK	FK principales	Propósito
-------	----	----------------	-----------

Tabla	PK	FK principales	Propósito
usuarios	id_usuario	-	Cuenta, nombre, correo, password_hash, rol y estado.
estudiantes	id_estudiante	id_usuario -> usuarios	Datos académicos del alumno.
profesores	id_profesor	id_usuario -> usuarios	Datos del docente.
cursos	id_curso	id_profesor -> profesores	Curso, sección, periodo, modalidad y aula.
matriculas	id_matricula	id_estudiante, id_curso	Relación alumno-curso.
horarios	id_horario	id_curso	Día, hora inicio/fin y aula.
sesiones_asistencia	id_sesion	id_curso	Clase concreta en la que se toma asistencia.
asistencias	id_asistencia	id_sesion, id_estudiante	Presente/tardanza/ausente y datos faciales.
datos_faciales	id_facial	id_estudiante	Embedding/plantilla biométrica autorizada.
chatbot_historial	id_chat	id_usuario	Historial opcional del asistente.
reportes_generados	id_reporte	id_usuario	Audita archivos generados.
auditoria	id_evento	id_usuario	Registra acciones críticas.

7.3 Script SQL completo

```
USE monitor_asistencia_ia;
```

```
CREATE TABLE IF NOT EXISTS usuarios (
  id_usuario BIGINT AUTO_INCREMENT PRIMARY KEY,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  correo VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  rol ENUM('ESTUDIANTE', 'PROFESOR', 'ADMINISTRADOR') NOT NULL,
  estado BOOLEAN NOT NULL DEFAULT TRUE,
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS estudiantes (
  id_estudiante BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_usuario BIGINT NOT NULL UNIQUE,
  codigo_estudiante VARCHAR(30) NOT NULL UNIQUE,
  carrera VARCHAR(120),
  ciclo VARCHAR(20),
  CONSTRAINT fk_est_usuario FOREIGN KEY (id_usuario)
  REFERENCES usuarios(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS profesores (
  id_profesor BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```

id_usuario BIGINT NOT NULL UNIQUE,
codigo_profesor VARCHAR(30) NOT NULL UNIQUE,
especialidad VARCHAR(120),
CONSTRAINT fk_prof_usuario FOREIGN KEY (id_usuario)
REFERENCES usuarios(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS cursos (
id_curso BIGINT AUTO_INCREMENT PRIMARY KEY,
codigo VARCHAR(30) NOT NULL UNIQUE,
nombre VARCHAR(150) NOT NULL,
seccion VARCHAR(30),
periodo_academico VARCHAR(20),
modalidad ENUM('PRESENCIAL','VIRTUAL','HIBRIDO') DEFAULT 'PRESENCIAL',
aula VARCHAR(50),
id_profesor BIGINT,
estado BOOLEAN DEFAULT TRUE,
CONSTRAINT fk_curso_prof FOREIGN KEY (id_profesor)
REFERENCES profesores(id_profesor) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS matriculas (
id_matricula BIGINT AUTO_INCREMENT PRIMARY KEY,
id_estudiante BIGINT NOT NULL,
id_curso BIGINT NOT NULL,
fecha_matricula TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
estado BOOLEAN DEFAULT TRUE,
UNIQUE KEY uq_matricula (id_estudiante,id_curso),
CONSTRAINT fk_mat_est FOREIGN KEY (id_estudiante)
REFERENCES estudiantes(id_estudiante) ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_mat_curso FOREIGN KEY (id_curso)
REFERENCES cursos(id_curso) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS horarios (
id_horario BIGINT AUTO_INCREMENT PRIMARY KEY,
id_curso BIGINT NOT NULL,
dia_semana ENUM('LUNES','MARTES','MIERCOLES','JUEVES','VIERNES','SABADO','DOMINGO') NOT NULL,
hora_inicio TIME NOT NULL,
hora_fin TIME NOT NULL,
aula VARCHAR(50),
CONSTRAINT fk_hor_curso FOREIGN KEY (id_curso)
REFERENCES cursos(id_curso) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS sesiones_asistencia (
id_sesion BIGINT AUTO_INCREMENT PRIMARY KEY,
id_curso BIGINT NOT NULL,
fecha DATE NOT NULL,
hora_inicio TIME NOT NULL,
hora_fin TIME,
estado ENUM('PROGRAMADA','ACTIVA','CERRADA') DEFAULT 'PROGRAMADA',
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_sesion_curso FOREIGN KEY (id_curso)

```

```

REFERENCES cursos(id_curso) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS asistencias (
  id_asistencia BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_sesion BIGINT NOT NULL,
  id_estudiante BIGINT NOT NULL,
  estado ENUM('PRESENTE','TARDANZA','AUSENTE') NOT NULL,
  hora_registro TIME,
  metodo_registro ENUM('FACIAL','MANUAL') DEFAULT 'FACIAL',
  confianza_facial DECIMAL(5,4),
  liveness BOOLEAN DEFAULT FALSE,
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_asistencia (id_sesion,id_estudiante),
  CONSTRAINT fk_asis_sesion FOREIGN KEY (id_sesion)
    REFERENCES sesiones_asistencia(id_sesion) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_asis_est FOREIGN KEY (id_estudiante)
    REFERENCES estudiantes(id_estudiante) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS datos_faciales (
  id_facial BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_estudiante BIGINT NOT NULL UNIQUE,
  embedding JSON,
  ruta_imagen_referencia VARCHAR(255),
  activo BOOLEAN DEFAULT TRUE,
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_facial_est FOREIGN KEY (id_estudiante)
    REFERENCES estudiantes(id_estudiante) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS chatbot_historial (
  id_chat BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_usuario BIGINT NOT NULL,
  mensaje TEXT NOT NULL,
  respuesta TEXT,
  intencion VARCHAR(60),
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_chat_usuario FOREIGN KEY (id_usuario)
    REFERENCES usuarios(id_usuario) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS reportes_generados (
  id_reporte BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_usuario BIGINT NOT NULL,
  tipo VARCHAR(50) NOT NULL,
  formato ENUM('XLSX','PDF') NOT NULL,
  filtros JSON,
  ruta_archivo VARCHAR(255),
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_rep_usuario FOREIGN KEY (id_usuario)
    REFERENCES usuarios(id_usuario) ON DELETE CASCADE
) ENGINE=InnoDB;

```

```
CREATE TABLE IF NOT EXISTS auditoria (
  id_evento BIGINT AUTO_INCREMENT PRIMARY KEY,
  id_usuario BIGINT NULL,
  accion VARCHAR(120) NOT NULL,
  entidad VARCHAR(80),
  entidad_id VARCHAR(80),
  detalle JSON,
  ip_origen VARCHAR(45),
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_aud_usuario FOREIGN KEY (id_usuario)
    REFERENCES usuarios(id_usuario) ON DELETE SET NULL
) ENGINE=InnoDB;
```

8. Conexión FastAPI ↔ MySQL

8.1 Archivo .env

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=monitor_asistencia_ia
DB_USER=app_asistencia
DB_PASSWORD=CAMBIAR_EN_LOCAL

SECRET_KEY=CAMBIAR_POR_UNA_CLAVE_LARGA_Y_ALEATORIA
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

SEGURIDAD No usar root desde la aplicación. Cree un usuario MySQL exclusivo para FastAPI con permisos solo sobre esta base. Nunca suba .env a GitHub.

8.2 Crear usuario MySQL para la aplicación

```
CREATE USER IF NOT EXISTS 'app_asistencia'@'localhost'
IDENTIFIED BY 'TU_PASSWORD_FUERTE';

GRANT SELECT, INSERT, UPDATE, DELETE
ON monitor_asistencia_ia.*
TO 'app_asistencia'@'localhost';

FLUSH PRIVILEGES;
```

8.3 database.py

```
import os
```

```

from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from urllib.parse import quote_plus

load_dotenv()

DB_USER = os.getenv("DB_USER")
DB_PASSWORD = quote_plus(os.getenv("DB_PASSWORD", ""))
DB_HOST = os.getenv("DB_HOST", "127.0.0.1")
DB_PORT = os.getenv("DB_PORT", "3306")
DB_NAME = os.getenv("DB_NAME", "monitor_asistencia_ia")

DATABASE_URL = (
    f"mysql+pymysql://{DB_USER}:{DB_PASSWORD}"
    f"@{DB_HOST}:{DB_PORT}/{DB_NAME}?charset=utf8mb4"
)

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

8.4 Endpoint de prueba

```

from sqlalchemy import text
from app.core.database import engine

@app.get("/api/test-db")
def test_db():
    with engine.connect() as conn:
        name = conn.execute(text("SELECT DATABASE()")).scalar()
    return {"status": "ok", "database": name}

```

Levante la API y pruebe GET /api/test-db desde Swagger. La respuesta esperada es status ok y database monitor_asistencia_ia.

9. Autenticación con JWT y contraseñas seguras

En MySQL la aplicación administra las cuentas. La contraseña se transforma en un hash antes de almacenarse. Nunca se guarda ni se devuelve la contraseña original.

9.1 security.py

```
from datetime import datetime, timedelta, timezone
from jose import jwt
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(password: str, password_hash: str) -> bool:
    return pwd_context.verify(password, password_hash)

def create_access_token(data: dict, secret_key: str, algorithm: str, minutes: int):
    payload = data.copy()
    payload["exp"] = datetime.now(timezone.utc) + timedelta(minutes=minutes)
    return jwt.encode(payload, secret_key, algorithm=algorithm)
```

9.2 Flujo de login

```
POST /api/auth/login
{ "correo": "usuario@dominio.com", "password": "..."} }
```

1. Buscar usuarios.correo.
2. Validar estado = TRUE.
3. verify_password(password, password_hash).
4. Crear JWT con id_usuario y rol.
5. Devolver access_token + perfil mínimo.
6. El frontend guarda el token y redirige por rol.

9.3 Creación del primer administrador

No inserte una contraseña en texto plano. Use un pequeño script Python para generar el hash y crear el administrador.

```
# backend/scripts/create_admin.py
from app.core.database import SessionLocal
from app.core.security import hash_password
from app.models.user import Usuario

with SessionLocal() as db:
    admin = Usuario(
        nombres="Administrador",
        apellidos="Sistema",
        correo="admin@asistencia.local",
        password_hash=hash_password("CAMBIAR_ESTA_PASSWORD"),
        rol="ADMINISTRADOR",
        estado=True,
    )
```

```
db.add(admin)
db.commit()
```

10. API y backend por módulos

Ruta	Método	Permiso	Uso
/api/auth/login	POST	Público	Iniciar sesión.
/api/auth/me	GET	Autenticado	Perfil del usuario actual.
/api/users/create	POST	Profesor/Admin	Crear cuentas según rol.
/api/courses	GET	Autenticado	Cursos filtrados por usuario.
/api/attendance/session	POST	Profesor/Admin	Crear o iniciar sesión de asistencia.
/api/attendance/register	POST	Profesor/Admin/servicio facial	Registrar asistencia validada.
/api/chatbot/message	POST	Autenticado	Pregunta contextual.
/api/reports/attendance	POST	Autenticado	Generar reporte respetando scope.

10.1 Dependencia get_current_user

```
Bearer JWT -> decodificar -> id_usuario -> SELECT usuarios
-> validar estado -> devolver identidad + rol
```

```
require_roles("ADMINISTRADOR", "PROFESOR")
-> si rol no está permitido: HTTP 403
```

11. Frontend por rol

11.1 Estudiante

- Mis cursos.
- Calendario.
- Mi asistencia, historial y estadísticas.
- Mensajes/calificaciones si se mantienen en alcance.
- Asistente IA limitado a su cuenta.
- No puede crear usuarios ni consultar otros alumnos.

11.2 Profesor

- Cursos asignados.

- Estudiantes de esos cursos.
- Tomar asistencia y abrir sesiones.
- Crear estudiantes.
- Estadísticas y alertas de sus cursos.
- Reportes de sus cursos.
- Chatbot sin acceso a información administrativa.

11.3 Administrador

- Visión global de usuarios, profesores, estudiantes, cursos y asistencia.
- Crear estudiantes y profesores.
- Configurar periodos, organización, branding y parámetros.
- Analítica global, gráficos y reportes.
- Auditoría y registros de actividad.
- Puede consultar gestión autorizada, nunca contraseñas, tokens ni secretos.

12. Módulo de asistencia

12.1 Flujo

```
Profesor -> selecciona curso -> abre sesión ACTIVA
-> cámara / registro manual
-> validar matrícula
-> evitar duplicado (id_sesion + id_estudiante)
-> determinar PRESENTE/TARDANZA/AUSENTE
-> INSERT asistencias
-> dashboard y estadísticas se actualizan
```

- La sesión debe estar ACTIVA.
- El estudiante debe estar matriculado en el curso.
- No se admite más de una asistencia por alumno y sesión.
- El profesor solo puede operar sobre sus cursos.
- El administrador puede intervenir por procedimiento autorizado y auditado.

13. Reconocimiento facial y liveness

La autenticación facial y el registro de asistencia facial deben tratarse como un servicio especializado. El navegador captura imagen/video; FastAPI procesa o delega el análisis; MySQL guarda el resultado autorizado y la referencia del embedding.

```
Captura -> detección facial -> alineación -> liveness
-> embedding -> comparación con datos_faciales
-> score de similitud -> validación de matrícula/sesión
-> INSERT asistencias
```

- Preferir embeddings sobre fotos crudas cuando sea viable.

- Calibrar el umbral facial con datos autorizados.
- Registrar confianza_facial y liveness.
- Definir consentimiento, finalidad, retención y eliminación de datos biométricos.
- No ejecutar entrenamiento pesado dentro de cada request.

14. Chatbot IA tipo Gemini y restricciones

El chatbot debe tener botón circular flotante, panel lateral derecho, modo pantalla completa, historial de conversación, sugerencias, tablas, gráficos y archivos. La experiencia puede ser similar a Gemini, pero la autorización siempre pertenece a la aplicación.

14.1 Archivos

Archivo	Responsabilidad
frontend/components/chatbot/chatbot.html	Botón, panel, mensajes, input y controles.
frontend/assets/css/chatbot.css	Panel lateral, responsive y fullscreen.
frontend/assets/js/chatbot.js	Enviar mensajes, mostrar respuestas y manejar UX.
backend/app/api/chatbot.py	Endpoint POST /api/chatbot/message.
services/chatbot_policy_service.py	Decide qué puede consultar el usuario.
services/chatbot_data_service.py	Ejecuta consultas SQL parametrizadas.
services/chatbot_llm_service.py	Redacta/explica con LLM opcional.
services/report_service.py	Genera Excel/PDF.

14.2 Flujo seguro

```

Texto/Voz -> JWT -> usuario + rol -> clasificar intención
-> construir scope -> ejecutar herramienta permitida
-> consulta SQL parametrizada -> resultado autorizado
-> redactar respuesta -> texto / voz / archivo

```

REGLA CRÍTICA El LLM no recibe acceso directo a MySQL y no debe ejecutar SQL libre generado por el modelo. FastAPI decide la intención, el scope y la consulta permitida; el modelo solo recibe resultados ya autorizados.

14.3 Restricciones por rol

Rol	Puede consultar	Debe denegarse
-----	-----------------	----------------

Rol	Puede consultar	Debe denegarse
Estudiante	Sus cursos, asistencia, faltas, tardanzas, horario, estadísticas y reporte personal.	Otro estudiante, datos privados del profesor, administrador, datos globales.
Profesor	Sus cursos, alumnos matriculados, asistencia, alertas y reportes de sus cursos.	Alumnos ajenos, cursos ajenos, información de administrador, configuración global.
Administrador	Usuarios, cursos, asistencia, analítica, reportes y configuración autorizada.	Contraseñas, hashes, tokens, claves secretas y biometría cruda no necesaria.

14.4 Ejemplos

Pregunta	Rol	Resultado
¿Cuántas faltas tengo?	Estudiante	Permitida: filtra por su id_estudiante.
¿Cuántas faltas tiene María?	Estudiante	Denegada.
Alumnos con baja asistencia de Python	Profesor de Python	Permitida.
Alumnos de un curso que no imparto	Profesor	Denegada.
Contraseña del administrador	Cualquiera	Denegada siempre.
Reporte general de asistencia	Administrador	Permitida.

15. Reportes Excel/PDF desde el chatbot

Rol	Reportes permitidos
Estudiante	Reporte personal de asistencia, tardanzas, faltas y evolución.
Profesor	Reporte por curso/sección asignada, alumnos, tardanzas y ausencias.
Administrador	Reportes globales por periodo, curso, profesor, área, tendencias y auditoría autorizada.

"Genera reporte de asistencia de Python de septiembre"

- > intención REPORT
- > validar JWT y scope
- > SELECT parametrizado en MySQL
- > pandas/openpyxl o ReportLab
- > guardar archivo temporal
- > registrar reportes_generados
- > devolver URL/ruta segura de descarga

16. Asistente de voz

La voz es otra interfaz del mismo chatbot; no cambia permisos. El audio se transforma a texto, se procesa con el mismo endpoint y la respuesta puede leerse en voz alta.

```
Micrófono -> Speech-to-Text -> texto  
-> /api/chatbot/message -> respuesta  
-> Text-to-Speech -> altavoz
```

- Agregar botón de micrófono junto al textarea.
- Solicitar permiso solo cuando el usuario lo pulse.
- Usar SpeechRecognition/webkitSpeechRecognition si está disponible.
- Usar speechSynthesis para lectura.
- Mostrar estados: Escuchando, Procesando, Hablando, Detenido.

17. Estadística, analítica y Machine Learning

Indicador	Fórmula / criterio	Uso
% asistencia	$(\text{Presentes} + \text{tardanzas válidas}) / \text{sesiones} \times 100$	Dashboard y alertas
% ausencias	$\text{Ausentes} / \text{sesiones} \times 100$	Riesgo de inasistencia
% tardanzas	$\text{Tardanzas} / \text{sesiones} \times 100$	Puntualidad
Media	$\Sigma x / n$	Promedio por grupo
Mediana	Valor central	Robustez frente a extremos
Varianza	$\Sigma (x - \mu)^2 / n$	Dispersión
Desv. estándar	$\sqrt{\text{Varianza}}$	Variabilidad
Tendencia	Comparación semanal/mensual	Evolución temporal

Un modelo predictivo puede estimar probabilidad de ausencia o tardanza futura usando historial, frecuencia reciente, patrón semanal y variables autorizadas. Debe mostrarse como estimación, no como certeza.

18. Adaptación a empresas u otros rubros

Concepto base	Educación	Empresa
Persona	Estudiante	Colaborador
Responsable	Profesor	Supervisor/Jefe
Grupo	Curso/Sección	Área/Departamento
Sesión	Clase	Turno/Jornada

Concepto base	Educación	Empresa
Asistencia	Asistencia	Marcación
Aula	Aula	Sede/Ubicación

El administrador puede seleccionar el tipo de organización y cambiar terminología, logo, colores, módulos visibles y formato de reportes sin alterar la lógica central de permisos y asistencia.

19. Puesta en marcha completa paso a paso

1. Crear la carpeta `monitor_asistencia_ia` y abrirla en VS Code.
2. Crear `.env` y activarlo.
3. Crear backend, frontend, database y ai con la estructura indicada.
4. Instalar FastAPI, Uvicorn, SQLAlchemy, PyMySQL, dotenv, Pydantic, JWT y bcrypt.
5. Instalar MySQL Server y Workbench.
6. Crear `monitor_asistencia_ia` con `utf8mb4`.
7. Ejecutar `database/02_schema.sql`.
8. Crear el usuario MySQL `app_asistencia` y otorgar permisos sobre la base.
9. Crear `backend/.env`.
10. Implementar `core/database.py` y probar la conexión.
11. Implementar modelos SQLAlchemy o consultas parametrizadas.
12. Implementar `security.py` para hash y JWT.
13. Crear primer administrador con script, nunca con password plano.
14. Implementar `/api/auth/login` y `/api/auth/me`.
15. Implementar dependencia `get_current_user` y `require_roles`.
16. Implementar `/api/users/create` con reglas: profesor solo estudiante; admin estudiante/profesor.
17. Conectar login y dashboards mediante fetch y Authorization Bearer.
18. Implementar cursos, matrículas, horarios, sesiones y asistencias.
19. Integrar chatbot básico, luego política por rol y consultas reales.
20. Agregar generación de reportes, voz y reconocimiento facial por fases.
21. Ejecutar pruebas de seguridad y respaldar la base.

19.1 Comandos mínimos

```
# activar entorno
.\.venv\Scripts\Activate.ps1
```

```
# backend
cd backend
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload

# Swagger
http://127.0.0.1:8000/docs
```

20. Pruebas funcionales y de seguridad

Prueba	Resultado esperado
Login estudiante	Solo dashboard estudiante.
Login profesor	Solo dashboard profesor.
Login administrador	Solo dashboard administrador.
Estudiante pregunta por otro alumno	Chatbot deniega.
Profesor consulta curso ajeno	HTTP 403 / chatbot deniega.
Profesor intenta crear profesor	HTTP 403.
Admin crea profesor	usuarios + profesores.
Profesor crea estudiante	usuarios + estudiantes y matrícula si corresponde.
JWT ausente/expirado	HTTP 401.
Usuario inactivo	HTTP 403.
Doble asistencia misma sesión	Rechazada por UNIQUE/validación.
Reporte fuera del scope	Denegado.
Rostro no autorizado	No registra asistencia.

21. Copias de seguridad y mantenimiento MySQL

- Programar backups periódicos.
- No respaldar únicamente archivos del proyecto; la base requiere respaldo independiente.
- Probar restauración en un entorno de prueba.
- Rotar credenciales y restringir accesos.
- Revisar índices y consultas lentas cuando crezca el volumen.

```
# respaldo desde terminal (ejemplo)
mysqldump -u root -p monitor_asistencia_ia > backup_monitor.sql
```

```
# restauración
mysql -u root -p monitor_asistencia_ia < backup_monitor.sql
```

22. Errores frecuentes y soluciones

Error	Causa común	Solución
uvicorn no se reconoce	Uvicorn no está en PATH/entorno.	<code>python -m uvicorn app.main:app --reload</code>
Attribute app not found	main.py no define <code>app = FastAPI()</code> .	Revisar <code>backend/app/main.py</code> .
ModuleNotFoundError	Dependencia instalada en otro Python.	Activar <code>.venv</code> y reinstalar.
Access denied for user	Credenciales/host MySQL incorrectos.	Revisar <code>.env</code> y GRANT.
Can't connect to MySQL	Servicio detenido o puerto incorrecto.	Iniciar servicio; revisar 3306.
Duplicate entry	Correo/código/relación unique repetida.	Validar antes de insertar.
Foreign key constraint fails	FK apunta a fila inexistente.	Crear registros padre en orden.
401	JWT ausente/expirado.	Relogear y renovar sesión.
403	Rol/scope no autorizado.	Verificar permisos y relación con curso.

23. Hoja de ruta recomendada

Fase	Objetivo	Prioridad
1	Cerrar esquema MySQL, Auth JWT y creación por rol.	Alta
2	Conectar dashboards a datos reales.	Alta
3	Completar sesiones y asistencia manual.	Alta
4	Completar chatbot seguro por scope.	Alta
5	Reportes Excel/PDF.	Media-Alta
6	Asistente de voz.	Media
7	Reconocimiento facial + liveness.	Alta / especializada
8	Analítica y ML predictivo.	Media
9	Modo empresa/personalizado.	Media
10	Auditoría, hardening, pruebas y despliegue.	Alta

24. Anexos

24.1 requirements.txt recomendado

```
fastapi
uvicorn[standard]
sqlalchemy
pymysql
python-dotenv
pydantic[email]
pydantic-settings
python-jose[cryptography]
passlib[bcrypt]
python-multipart
pandas
openpyxl
reportlab
numpy
opencv-python
scikit-learn
joblib
httpx
pytest
# tensorflow/onnxruntime solo si el modelo lo necesita
```

24.2 .env.example

```
APP_ENV=development
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=monitor_asistencia_ia
DB_USER=app_asistencia
DB_PASSWORD=CAMBIAR_EN_LOCAL
SECRET_KEY=CAMBIAR_POR_CLAVE_SEGURA
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
MODEL_PATH=./ai/attendance_model/model.joblib
FACE_MODEL_PATH=./ai/face/model/
FACE_THRESHOLD=0.60
```

24.3 Consultas de verificación

```
USE monitor_asistencia_ia;
SHOW TABLES;

SELECT u.id_usuario, u.nombres, u.apellidos, u.rol, u.estado
FROM usuarios u;

SELECT c.id_curso, c.nombre, p.id_profesor
FROM cursos c
```

```
LEFT JOIN profesores p ON p.id_profesor = c.id_profesor;

SELECT a.id_asistencia, a.estado, a.hora_registro,
       e.id_estudiante, s.id_sesion
FROM asistencias a
JOIN estudiantes e ON e.id_estudiante = a.id_estudiante
JOIN sesiones_asistencia s ON s.id_sesion = a.id_sesion
ORDER BY a.fecha_registro DESC;
```

24.4 Regla final del chatbot

ESTUDIANTE Solo consulta datos de su propia cuenta. No puede consultar a otro estudiante, al profesor como sujeto privado ni al administrador.

PROFESOR Consulta sus cursos y alumnos matriculados en ellos. Puede crear estudiantes y generar reportes de sus cursos. No accede a información administrativa ni a cursos ajenos.

ADMINISTRADOR Control global autorizado sobre estudiantes, profesores, cursos, asistencias, analítica, reportes y configuración. Nunca se exponen contraseñas, hashes, tokens o claves secretas.

FIN DEL MANUAL